

Centro Universitario de Ciencias Exactas e
Ingenierías (CUCEI)

SEMINARIO DE SOLUCION DE PROBLEMAS
DE TRADUCTORES DE LENGUAJES II

Etapa del proyecto analizador léxico
completo.

Mateo Sebastián serrano Pérez

25/01/2026



1. Introducción

En el presente proyecto se desarrolla un analizador léxico completo, el cual constituye la primera fase dentro del proceso de compilación y tiene como objetivo principal analizar el código fuente carácter por carácter para identificar los diferentes símbolos del lenguaje.

Durante este análisis, el código fuente es transformado en una secuencia ordenada de tokens o símbolos léxicos. Cada token representa una unidad significativa del lenguaje, como identificadores, números, cadenas, operadores o palabras reservadas. Estos tokens serán utilizados posteriormente por el analizador sintáctico para verificar la estructura del programa.

El analizador fue implementado en Python, utilizando clases para representar los tokens y el analizador léxico, con el fin de tener una estructura clara y organizada.

2. Objetivos

El objetivo principal del proyecto es implementar un analizador léxico funcional utilizando Python. Además, se busca que el programa reconozca correctamente identificadores, números enteros y reales, cadenas de texto, operadores y símbolos especiales, así como palabras reservadas como `if`, `while`, `return`, `else` y tipos de datos como `int`, `float` y `void`.

También se pretende aplicar un algoritmo basado en un bucle infinito (`while True`) junto con una estructura tipo `switch` utilizando `if / elif`, manejar errores léxicos y reportar la línea y columna de cada token reconocido para facilitar la depuración del código.

3. Descripción General del Analizador

El sistema está conformado principalmente por una enumeración llamada `TokenType`, una clase `Token` y la clase principal `AnalizadorLexico`.

La enumeración `TokenType` define todos los tipos de tokens que el lenguaje puede reconocer. La clase `Token` almacena la información de cada símbolo, como su tipo, valor, línea y columna.

La clase `AnalizadorLexico` se encarga de recorrer el código fuente. Para ello utiliza un bucle infinito (`while True`) que procesa el texto hasta llegar al final del archivo.

La identificación de los tokens se realiza mediante una serie de condiciones if / elif, las cuales funcionan como una estructura tipo switch.

Dependiendo del carácter leído, el analizador decide si debe reconocer un identificador, un número, una cadena, un operador o un símbolo especial. Para facilitar esta tarea, se implementaron funciones auxiliares como reconocer_identificador, reconocer_numero y reconocer_cadena.

4. Símbolos Léxicos Reconocidos

El programa es capaz de reconocer identificadores formados por letras y dígitos, números enteros y números reales, cadenas de texto delimitadas por comillas dobles, palabras reservadas, operadores aritméticos, relacionales, lógicos y de igualdad, así como símbolos especiales como paréntesis, llaves, comas, punto y coma y el operador de asignación.

5. Manejo de Errores

El analizador léxico incluye un mecanismo de detección de errores con el objetivo de identificar entradas inválidas. Entre los errores que pueden presentarse se encuentran caracteres no reconocidos por el lenguaje, operadores incompletos como & o |, y cadenas de texto que no cuentan con una comilla de cierre.

Cuando se detecta un error, el sistema genera un mensaje indicando el tipo de error junto con la línea y columna exacta donde ocurrió, lo cual facilita la corrección del código fuente.

6. Pruebas y Resultados

Para verificar el correcto funcionamiento del analizador léxico, se realizaron diversas pruebas utilizando archivos con fragmentos de código que incluyen declaraciones de variables, estructuras de control como if y while, operaciones aritméticas y expresiones lógicas.

En todos los casos, el programa mostró el código analizado y posteriormente imprimió una tabla con los tokens reconocidos, incluyendo su tipo, valor, línea y columna. También se reportaron los errores cuando fue necesario.

Los resultados obtenidos demuestran que el analizador cumple con los requisitos establecidos y produce salidas consistentes.

```
Ingresa el nombre del archivo a analizar: ejemplo.txt

=====
Archivo: ejemplo.txt
Código a analizar:
=====

int suma(int a, int b) {
    return a + b;
}

float promedio(float x, float y) {
    float resultado;
    resultado = (x + y) / 2.0;
    return resultado;
}

int main() {
    int numero1;
    int numero2;
    float valor1;
    float valor2;
    int resultado;

    numero1 = 10;
    numero2 = 20;
    valor1 = 3.14;
    valor2 = 2.5;

    if (numero1 < numero2) {
        resultado = suma(numero1, numero2);
    } else {
        resultado = 0;
    }

    while (numero1 > 0 && numero2 != 0) {
        numero1 = numero1 - 1;
        if (numero1 == 5 || numero2 >= 15) {
            break;
        }
    }

    if ((!(valor1 <= valor2)) {
        valor1 = valor1 * 2.0;
    }

    return 0;
}
```

TOKENS RECONOCIDOS:			
Tipo	Valor	Línea	Columna
TIPO	int	1	1
IDENTIFICADOR	suma	1	5
PARENTESIS_ABRE	(	1	9
TIPO	int	1	10
IDENTIFICADOR	a	1	14
COMA	,	1	15
TIPO	int	1	17
IDENTIFICADOR	b	1	21
PARENTESIS_CIERRA	)	1	22
LLAVE_ABRE	{	1	24
RETURN	return	2	5
IDENTIFICADOR	a	2	12
OP_SUMA	+	2	14
IDENTIFICADOR	b	2	16
PUNTO_COMA	;	2	17
LLAVE_CIERRA	}	3	1
TIPO	float	5	1
IDENTIFICADOR	promedio	5	7
PARENTESIS_ABRE	(	5	15
TIPO	float	5	16
IDENTIFICADOR	x	5	22
COMA	,	5	23
TIPO	float	5	25
IDENTIFICADOR	y	5	31

7. Conclusión

Con lo realizado en este analizador léxico se cumplen los objetivos del proyecto, ya que el programa reconoce correctamente los símbolos definidos, maneja errores de manera adecuada y presenta una estructura clara.

Este analizador puede ser utilizado como base para la implementación de un analizador sintáctico en etapas posteriores del desarrollo del compilador.